جامعة الأمير سطام بن عبد العزيز
Prince Sattam Bin Abdulaziz University

CCES – College of Computer Engineering and Sciences

CS3401 – Algorithm Design and Analysis

Chapter 7-Dynamic programming

References:

- *Introduction to the Design and Analysis of Algorithms. Anany V. Levitin, Pearson 3rd Edition.*

Dynamic programming

Introduction

- Dynamic Programming is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems.

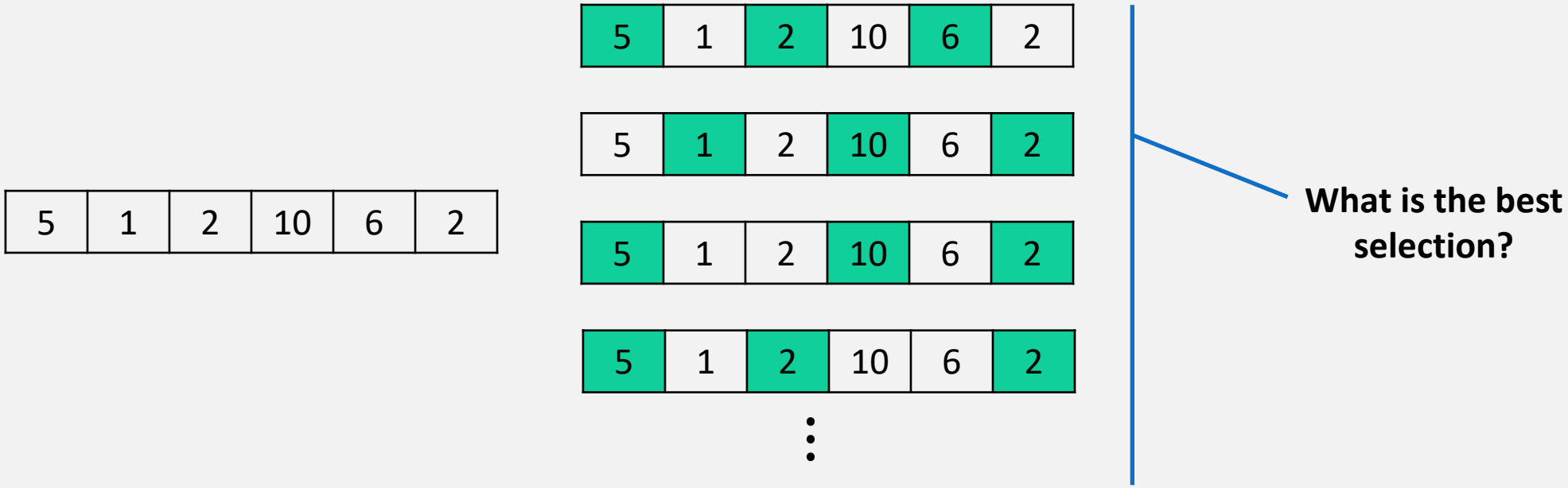
"Programming" here means "planning"

- Main idea:
  - set up a recurrence relating a solution to a larger instance to solutions of some smaller instances
  - solve smaller instances once
  - record solutions in a table
  - extract solution to the initial instance from that table

Dynamic programming

Example 1: Coin-row problem

- There is a row of n coins whose values are some positive integers $c_1, c_2,\ldots,c_n$, not necessarily distinct. The goal is to pick up the maximum amount of money subject to the constraint that no two coins adjacent in the initial row can be picked up.



Dynamic programming

Example 1: Coin-row problem

- Let F(n) be the maximum amount that can be picked up from the row of n coins. To derive a recurrence for F(n), we partition all the allowed coin selections into two groups:
  - Those without last coin – the max amount is ?
  - Those with the last coin -- the max amount is ?

Thus we have the following recurrence

$$F(n) = \max\{c_n + F(n-2), F(n-1)\} \text{ for } n > 1,$$

$$F(0) = 0, F(1) = c_1.$$

index	0	1	2	3	4	5	6
coins	--	5	1	2	10	6	2
F( )							

Max amount:

optimal solution:

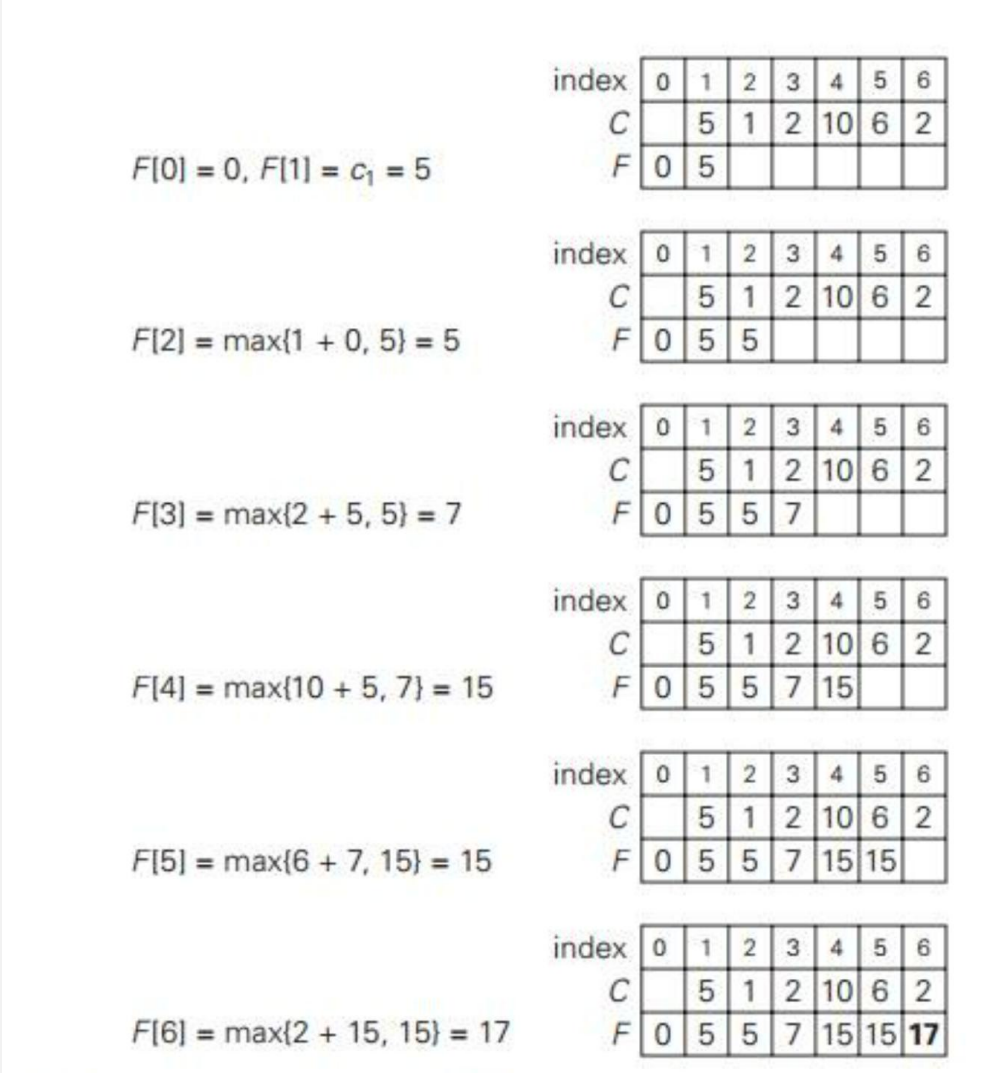
Time efficiency:

Space efficiency:

Dynamic programming

Example 1: Coin-row problem

- The optimal solution is 17 which represents the maximum amount collected.
- The coins that contributes in the optimal solutions are C1 = 5, C4 = 10, and C6 = 2.
- The complexity of finding the optimal solution is O(n).



Dynamic programming

Example 2: Change-making problem

- Find the minimum number of coins of denominations $d_1 < d_2 < \ldots < d_m$ needed to make change for amount n.

GreedyChange(money)

Change ← empty collection of coins
while money > 0:
coin ← largest denomination that does not exceed money
add coin to Change
money ← money − coin
return Change

US Money

40 cents = 25 + 10 + 5

Dynamic programming

Example 2: Change-making problem

- Find the minimum number of coins of denominations $d_1 < d_2 < \ldots < d_m$ needed to make change for amount n.

GreedyChange(money)

Change ← empty collection of coins
while money > 0:
coin ← largest denomination that does not exceed money
add coin to Change
money ← money − coin
return Change

Tanzania Money

40 cents = 25 + 10 + 5

Optimal solution

40 cents = 20 + 20

Dynamic programming

Example 2: Change-making problem

- Find the minimum number of coins of denominations $d_1 < d_2 < \ldots < d_m$ needed to make change for amount n.

$$F(n) = \min_{j:n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{for } n > 0,$$

$$F(0) = 0.$$

ALGORITHM *ChangeMaking*(*D*[1..*m*], *n*)
F[0]←0
for *i* ←1 **to** *n* **do**
temp←∞; *j* ←1
while *j* ≤ *m* **and** *i* ≥ *D*[*j*] **do**
temp ←min(*F* [*i* − *D*[*j*]], *temp*)
j ←*j* + 1
F[*i*]←*temp* + 1
return *F*[*n*]

Dynamic programming

Example 2: Change-making problem

The application of the algorithm to amount n = 6 and denominations 1, 3, 4

$$F(n) = \min_{j:n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{for } n > 0,$$

$$F(0) = 0.$$

To find the coins of an optimal solution, we need to back-trace the computations to see which of the denominations produced the minima

Time efficiency O(nm)
Space efficiency θ(n)

$F[0] = 0$

n	0	1	2	3	4	5	6
F	0						

$F[1] = \min\{F[1-1]\} + 1 = 1$

n	0	1	2	3	4	5	6
F	0	1					

$F[2] = \min\{F[2-1]\} + 1 = 2$

n	0	1	2	3	4	5	6
F	0	1	2				

$F[3] = \min\{F[3-1], F[3-3]\} + 1 = 1$

n	0	1	2	3	4	5	6
F	0	1	2	1			

$F[4] = \min\{F[4-1], F[4-3], F[4-4]\} + 1 = 1$

n	0	1	2	3	4	5	6
F	0	1	2	1	1		

$F[5] = \min\{F[5-1], F[5-3], F[5-4]\} + 1 = 2$

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	

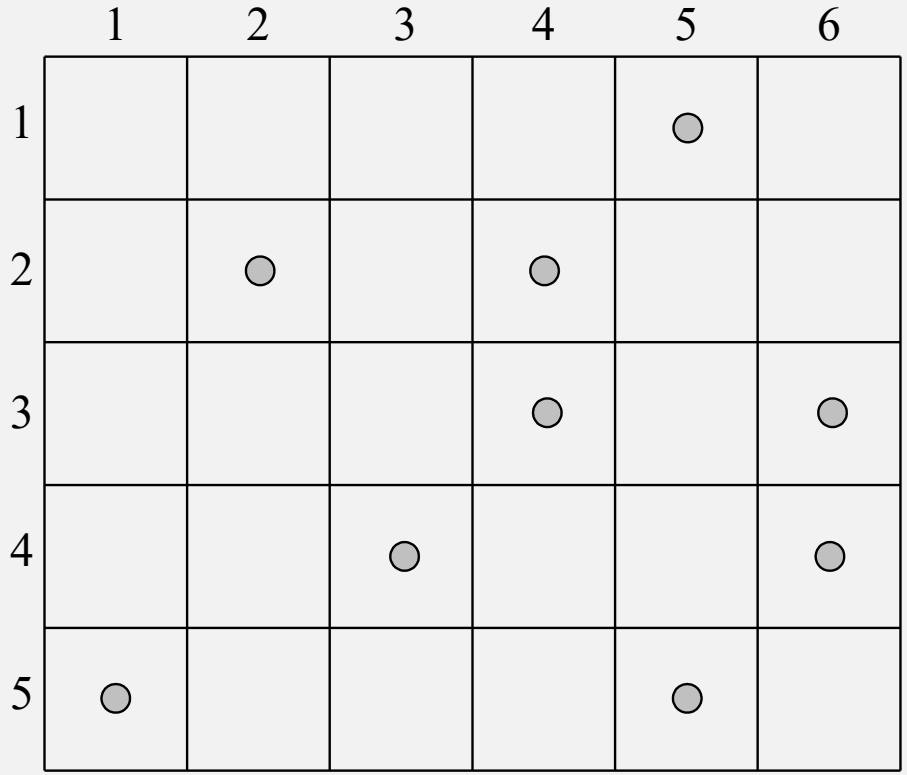
$F[6] = \min\{F[6-1], F[6-3], F[6-4]\} + 1 = 2$

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	**2**

Dynamic programming

Example 3: Coin-collecting problem

- Several coins are placed in cells of an n × m board, no more than one coin per cell. A robot, located in the upper left cell of the board, needs to collect as many of the coins as possible and bring them to the bottom right cell. On each step, the robot can move either one cell to the right or one cell down from its current location.



Dynamic programming

Example 3: Coin-collecting problem

- Let F(*i*,*j*) be the largest number of coins the robot can collect and bring to cell (*i*,*j*) in the *i*th row and *j*th column.
- The largest number of coins that can be brought to cell (*i*,*j*):
  - from the left neighbor ?
  - from the neighbor above?
- The recurrence:

$$F(i, j) = \max\{F(i-1, j),\ F(i, j-1)\} + c_{ij} \quad \text{for } 1 \le i \le n,\ 1 \le j \le m$$

where $c_{ij} = 1$ if there is a coin in cell (i,j), and $c_{ij} = 0$ otherwise

$F(0, j) = 0$ for $1 \le j \le m$ and $F(i, 0) = 0$ for $1 \le i \le n$.

Dynamic programming

Example 3: Coin-collecting problem

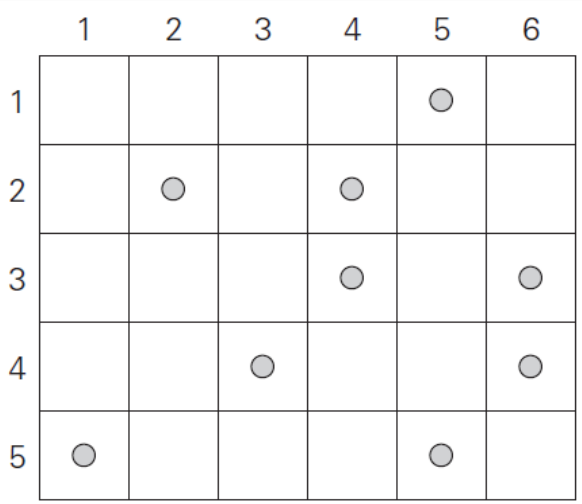
$F(i, j) = \max\{F(i-1, j),\ F(i, j-1)\} + c_{ij}$ for $1 \le i \le n$, $1 \le j \le m$

where $c_{ij} = 1$ if there is a coin in cell (i,j),

and $c_{ij} = 0$ otherwise

$F(0, j) = 0$ for $1 \le j \le m$

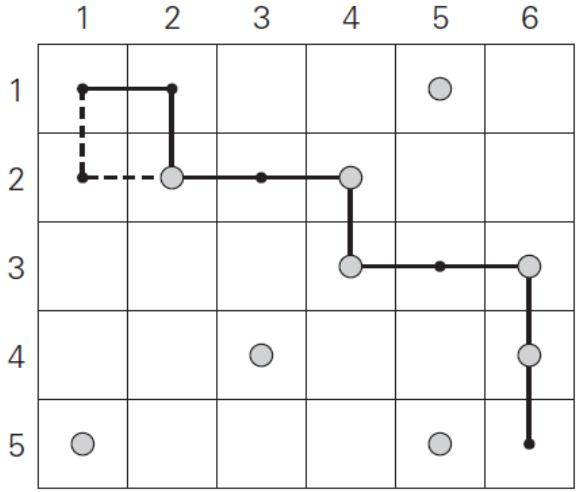
$F(i, 0) = 0$ for $1 \le i \le n$.



(a)

	1	2	3	4	5	6
1	0	0	0	0	1	1
2	0	1	1	2	2	2
3	0	1	1	3	3	4
4	0	1	2	3	3	5
5	1	1	2	3	4	**5**

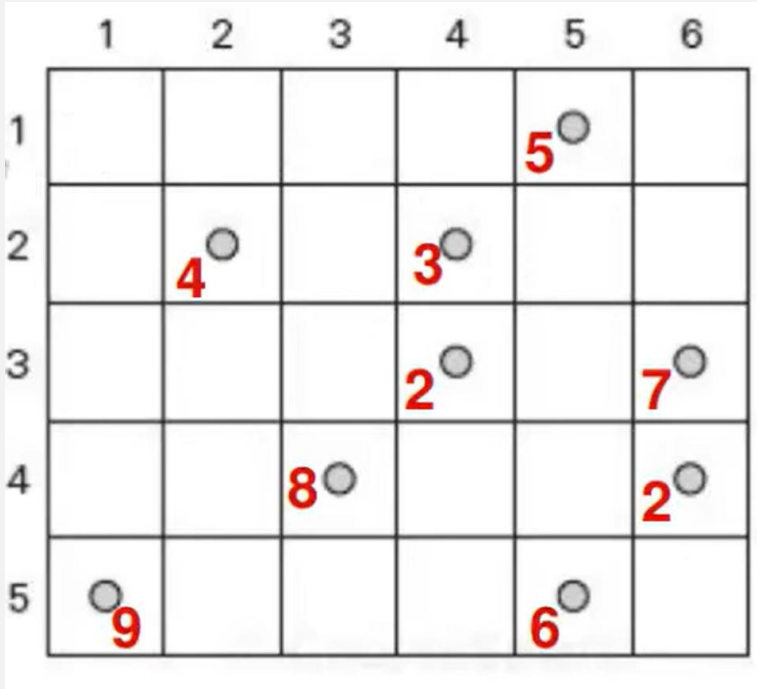
(b)



Dynamic programming

Example 3: Coin-collecting problem

Here, the same problem can have some values of the coin collected instead of being 1 or 0, as in the previous example.



	1	2	3	4	5	6
1	0	0	0	0	5	5
2	0	4	4	7	7	7
3	0	4	4	9	9	16
4	0	4	12	12	12	18
5	9	9	12	12	18	18

Dynamic programming

Knapsack problem

- given n items of known weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ and a knapsack of capacity W, find the most valuable subset of the items that fit into the knapsack.
- Let us consider an instance defined by the first i items, $1 \le i \le n$, with weights $w_1, \ldots, w_i$, values $v_1, \ldots, v_i$, and knapsack capacity j, $1 \le j \le W$.
- Let $F(i, j)$: The value of the most valuable subset of the first i items that fit into the knapsack of capacity j.

$$F(i, j) = \begin{cases} \max\{F(i-1, j), v_i + F(i-1, j-w_i)\} & \text{if } j - w_i \ge 0, \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

$F(0, j) = 0$ for $j \ge 0$ and $F(i, 0) = 0$ for $i \ge 0$

Dynamic programming

Knapsack problem - Example

Knapsack of capacity $W = 5$

item	weight	value
1	2	$12
2	1	$10
3	3	$20
4	2	$15

$$F(i, j) = \begin{cases} \max\{F(i-1, j),\ v_i + F(i-1, j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

$F(0, j) = 0$ for $j \geq 0$ and $F(i, 0) = 0$ for $i \geq 0$

		Capacity j					
		0	1	2	3	4	5
	0						
$w_1 = 2, v_1 = 12$	1						
$w_2 = 1, v_2 = 10$	2						
$w_3 = 3, v_3 = 20$	3						
$w_4 = 2, v_4 = 15$	4						

Optimal solution

Dynamic programming

Knapsack problem - Example

Knapsack of capacity $W = 5$

item	weight	value
1	2	$12
2	1	$10
3	3	$20
4	2	$15

$$F(i, j) = \begin{cases} \max\{F(i-1, j), v_i + F(i-1, j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

$F(0, j) = 0$ for $j \geq 0$ and $F(i, 0) = 0$ for $i \geq 0$

		Capacity j					
		0	1	2	3	4	5
	0	0	0	0	0	0	0
$w_1 = 2, v_1 = 12$	1	0	0	12	12	12	12
$w_2 = 1, v_2 = 10$	2	0	10	12	22	22	22
$w_3 = 3, v_3 = 20$	3	0	10	12	22	30	32
$w_4 = 2, v_4 = 15$	4	0	10	15	25	30	37

Optimal solution

Items 1-2-4

- The time efficiency and space efficiency $\theta(nW)$.
- The time to find the composition of an optimal solution is $O(n)$.

Dynamic programming

Knapsack problem – Example with memory function

Knapsack of capacity $W = 5$

item	weight	value
1	2	$12
2	1	$10
3	3	$20
4	2	$15

$$F(i, j) = \begin{cases} \max\{F(i-1, j),\ v_i + F(i-1, j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

$F(0, j) = 0$ for $j \geq 0$ and $F(i, 0) = 0$ for $i \geq 0$

		Capacity j					
		0	1	2	3	4	5
	0	0	0	0	0	0	0
$w_1 = 2, v_1 = 12$	1	0	?	?	?	?	?
$w_2 = 1, v_2 = 10$	2	0		?	?		?
$w_3 = 3, v_3 = 20$	3	0			?		?
$w_4 = 2, v_4 = 15$	4	0					?

Dynamic programming

Knapsack problem – Example with memory function

Knapsack of capacity $W = 5$

item	weight	value
1	2	$12
2	1	$10
3	3	$20
4	2	$15

$$F(i, j) = \begin{cases} \max\{F(i-1, j),\ v_i + F(i-1, j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

$F(0, j) = 0$ for $j \geq 0$ and $F(i, 0) = 0$ for $i \geq 0$

		Capacity j					
		0	1	2	3	4	5
	0	0	0	0	0	0	0
$w_1 = 2, v_1 = 12$	1	0	0	12	12	12	12
$w_2 = 1, v_2 = 10$	2	0		12	22		22
$w_3 = 3, v_3 = 20$	3	0			22		32
$w_4 = 2, v_4 = 15$	4	0					37

ALGORITHM *MFKnapsack(i, j)*
if $F[i, j] < 0$
if $j < Weights[i]$
$value \leftarrow MFKnapsack(i - 1, j)$
else
$value \leftarrow \max(MFKnapsack(i - 1, j),$
$Values[i] + MFKnapsack(i - 1, j - Weights[i]))$
$F[i, j] \leftarrow value$
return $F[i, j]$

Dynamic programming

Knapsack problem – Example with memory function

$$F(i, j) = \begin{cases} \max\{F(i-1, j),\ v_i + F(i-1, j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

$F(0, j) = 0$ for $j \geq 0$ and $F(i, 0) = 0$ for $i \geq 0$

Step	Expression	Explanation	Formula Used
1	**V[4,5]**, i=4, j=5, w_4=2, p_4=15	Since $w_4 \leq j \rightarrow 5$	V(4,5) = Max{ V(3,5), V(3,3) + 15 }
2	**V[3,5]**, i=3, j=5, w_3=3, p_3=20	Since $w_3 \leq j \rightarrow 5$	V(3,5) = Max{ V(2,5), V(2,2) + 20 }
3	**V[3,3]**, i=3, j=3, w_3=3, p_3=20	Since w_3 = j	V(3,3) = Max{ V(2,3), V(2,0) + 20 }
4	**V[2,5]**, i=2, j=5, w_2=1, p_2=10	Since $w_2 \leq j$	V(2,5) = Max{ V(1,5), V(1,4) + 10 }
5	**V[2,1]**, i=2, j=1, w_2=1, p_2=10	Since w_2 = j	V(2,1) = Max{ V(1,1), V(1,0) + 10 }
6	**V[2,3]**, i=2, j=3, w_2=1, p_2=10	Since $w_2 \leq j$	V(2,3) = Max{ V(1,3), V(1,2) + 10 }
7	**V[2,0]**, i=2, j=0, w_2=1, p_2=10	Since $w_2 > j$	V(2,0) = 0
8	**V[1,5]**, i=1, j=5, w_1=2, p_1=12	Since $w_1 \leq j$	V(1,5) = Max{ V(0,5), V(0,3) + 12 }
9	**V[1,4]**, i=1, j=4, w_1=2, p_1=12	Since $w_1 \leq j$	V(1,4) = Max{ V(0,4), V(0,2) + 12 }

		Capacity j					
		0	1	2	3	4	5
	0	0	0	0	0	0	0
$w_1 = 2, V_1 = 12$	1	0	0	12	12	12	12
$w_2 = 1, V_2 = 10$	2	0		12	22		22
$w_3 = 3, V_3 = 20$	3	0			22		32
$w_4 = 2, V_4 = 15$	4	0					37

Dynamic programming

Optimal Binary Search Trees

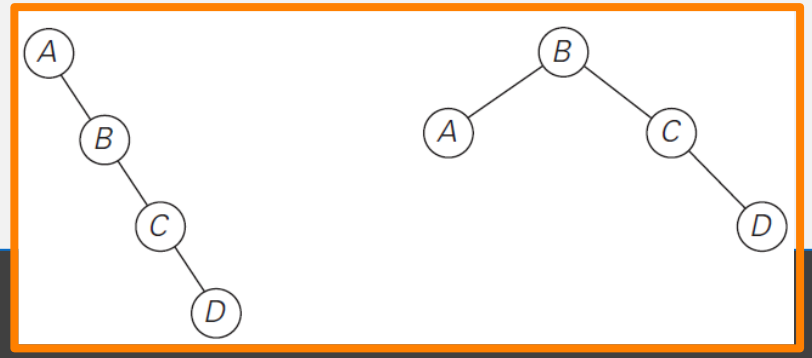
- Problem: Given n keys $a_1 < \ldots < a_n$ and probabilities $p_1 \leq \ldots \leq p_n$ searching for them, find a BST with a minimum average number of comparisons in successful search.

- Since total number of BSTs with n nodes is given by C(2n,n)/(n+1), which grows exponentially, brute force is hopeless.

Average number of comparisons
0.1*2+ 0.2*1 + 0.4*2+ 0.3*3 = 2.1

- Level of the root:1
- consider four keys A, B, C, and D to be searched for with probabilities 0.1, 0.2, 0.4, and 0.3, respectively **(What is the optimal search tree?)**

Average number of comparisons
0.1*1+ 0.2*2 + 0.4*3+ 0.3*4 = 2.9



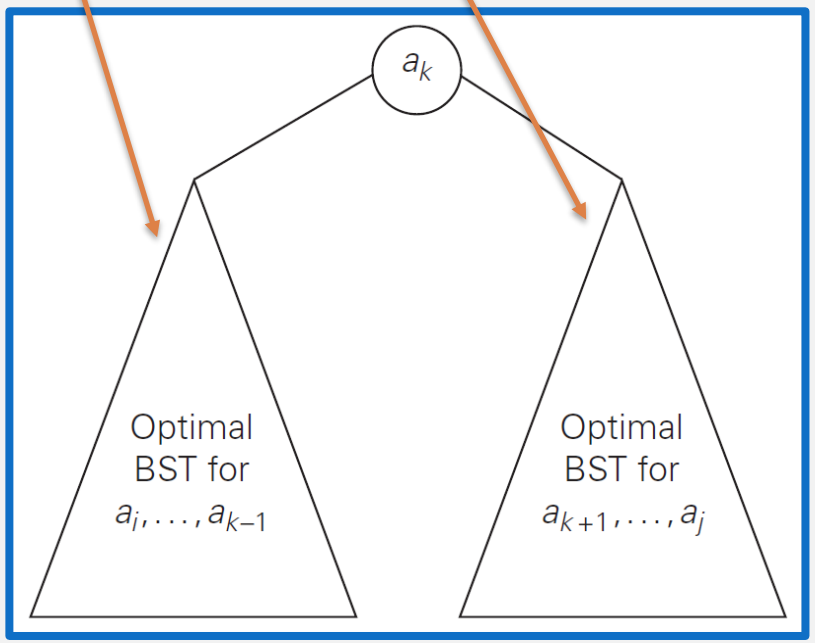
Dynamic programming

Optimal Binary Search Trees

- Let C[i,j] be the optimal number of comparisons for nodes i to j.

- $C[i,j] = min_{i \le k \le j} \begin{cases} C[i,k-1] + C[k+1,j] + \sum_{s=i}^{j} P_s \\ C[i,i-1] = 0 \end{cases}$

- $C[i,i] = P_i$



Dynamic programming

Optimal Binary Search Trees - Example

key	A	B	C	D
probability	0.1	0.2	0.4	0.3

i \ j	0	1	2	3	4
1	0				
2		0			
3			0		
4				0	
5					0

i \ j	0	1	2	3	4
1					
2					
3					
4					
5					

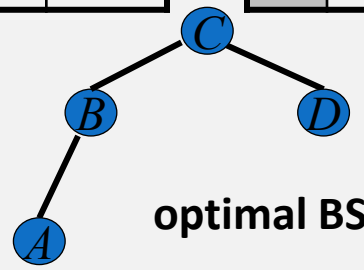
Dynamic programming

Optimal Binary Search Trees - Example

key	A	B	C	D
probability	0.1	0.2	0.4	0.3

i \ j	0	1	2	3	4
1	0	.1	.4	1.1	1.7
2		0	.2	.8	1.4
3			0	.4	1.0
4				0	.3
5					0

i \ j	0	1	2	3	4
1		1	2	3	3
2			2	3	3
3				3	3
4					4
5					



optimal BST

$$C[1,2] = \min \begin{cases} k=1: & C[1,0]+C[2,2]+\sum_{s=1}^{2} p_s = 0+0.2+0.3 = 0.5 \\ k=2: & C[1,1]+C[3,2]+\sum_{s=1}^{2} p_s = 0.1+0+0.3 = \mathbf{0.4} \end{cases} = 0.4$$

$$C[2,3] = \min \begin{cases} k=2: & C[2,1]+C[3,3]+\sum_{s=2}^{3} p_s = 0+0.4+0.6 = 1.0 \\ k=3: & C[2,2]+C[4,3]+\sum_{s=2}^{3} p_s = 0.2+0+0.6 = \mathbf{0.8} \end{cases} = 0.8$$

$$C[3,4] = \min \begin{cases} k=3: & C[3,2]+C[4,4]+\sum_{s=3}^{4} p_s = 0+0.3+0.7 = \mathbf{1.0} \\ k=4: & C[3,3]+C[5,4]+\sum_{s=3}^{4} p_s = 0.4+0+0.7 = 1.1 \end{cases} = 1.0$$

$$C[1,3] = \min \begin{cases} k=1: & C[1,0]+C[2,3]+\sum_{s=1}^{3} p_s = 0+0.8+0.7 = 1.5 \\ k=2: & C[1,1]+C[3,3]+\sum_{s=1}^{3} p_s = 0.1+0.4+0.7 = 1.2 \\ k=3: & C[1,2]+C[4,3]+\sum_{s=1}^{3} p_s = 0.4+0+0.7 = \mathbf{1.1} \end{cases} = 1.1$$

$$C[2,4] = \min \begin{cases} k=2: & C[2,1]+C[3,4]+\sum_{s=2}^{4} p_s = 0+1.0+0.9 = 1.9 \\ k=3: & C[2,2]+C[4,4]+\sum_{s=2}^{4} p_s = 0.2+0.3+0.9 = \mathbf{1.4} \\ k=4: & C[2,3]+C[5,4]+\sum_{s=2}^{4} p_s = 0.8+0+0.9 = 1.7 \end{cases} = 1.1$$

$$C[1,4] = \min \begin{cases} k=1: & C[1,0]+C[2,4]+\sum_{s=1}^{s=4} p_s = 0+1.4+1.0 = 2.4 \\ k=2: & C[1,1]+C[3,4]+\sum_{s=1}^{4} p_s = 0.1+1.0+1.0 = 2.1 \\ k=3: & C[1,2]+C[4,4]+\sum_{s=1}^{4} p_s = 0.4+0.3+1.0 = \mathbf{1.7} \\ k=4: & C[1,3]+C[5,4]+\sum_{s=1}^{4} p_s = 1.1+0+1.0 = 2.1 \end{cases} = 1.7$$

Dynamic programming

Optimal Binary Search Trees

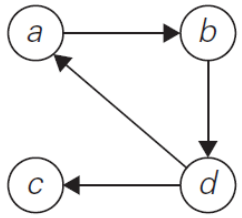
ALGORITHM *OptimalBST(P* [1..*n*]*)*
for *i* ←1 **to** *n* **do**
C[*i*, *i* − 1]←0
C[*i*, *i*]←*P*[*i*]
R[*i*, *i*]←*i*
C[*n* + 1, *n*]←0
for *d* ←1 **to** *n* − 1 **do** //diagonal count
for *i* ←1 **to** *n* − *d* **do**
j ←*i* + *d*
minval←∞
for *k*←*i* **to** *j* **do**
if *C*[*i*, *k* − 1]+ *C*[*k* + 1, *j*]< *minval*
minval←*C*[*i*, *k* − 1]+ *C*[*k* + 1, *j*]; *kmin*←*k*
R[*i*, *j*]←*kmin*
sum←*P*[*i*]; **for** *s* ←*i* + 1 **to** *j* **do** *sum*←*sum* + *P*[*s*]
C[*i*, *j*]←*minval* + *sum*
return *C*[1, *n*], *R*

Time efficiency: $\Theta(n^3)$
Space efficiency: $\Theta(n^2)$

Dynamic programming

Warshall's Algorithm: Transitive Closure

- The ***transitive closure*** of a directed graph with n vertices can be defined as the $n \times n$ boolean matrix $T = \{t_{ij}\}$, in which the element in the i^{th} row and the j^{th} column is 1 if there exists a nontrivial path (i.e., directed path of a positive length) from the i^{th} vertex to the j^{th} vertex; otherwise, t_{ij} is 0.



(a)

$A =$

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	0	1	0

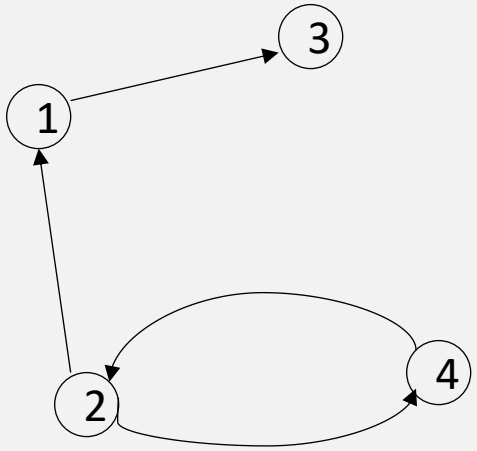
(b)

$T =$

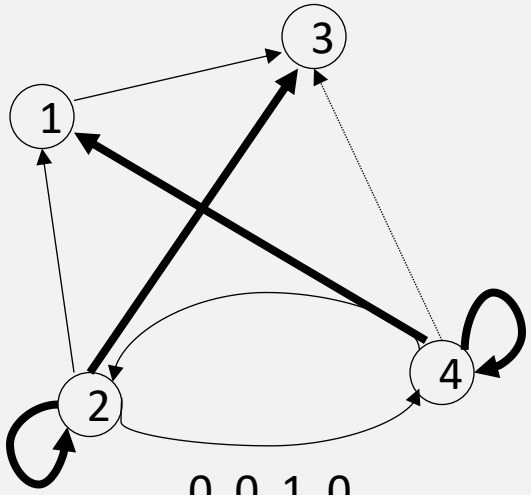
	a	b	c	d
a	1	1	1	1
b	1	1	1	1
c	0	0	0	0
d	1	1	1	1

(c)

(a) Digraph. (b) Its adjacency matrix. (c) Its transitive closure.



0 0 1 0
1 0 0 1
0 0 0 0
0 1 0 0



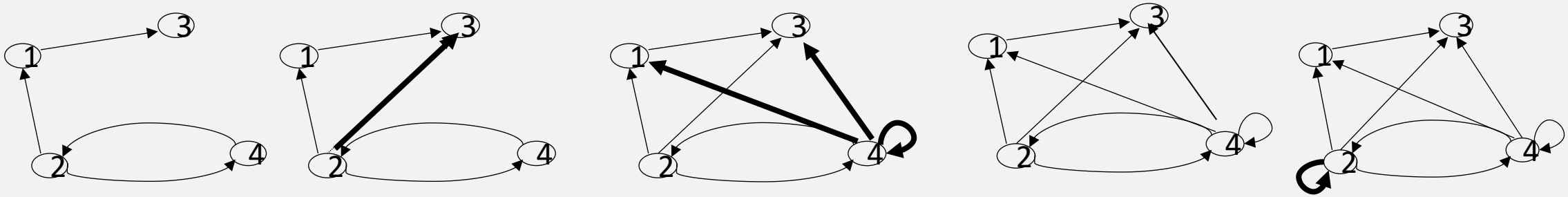
0 0 1 0
1 **1** **1** 1
0 0 0 0
1 1 **1** **1**

Dynamic programming

Warshall's Algorithm: Transitive Closure

Main idea: a path exists between two vertices i, j, if

- there is an edge from i to j; or
- there is a path from i to j going through vertex 1; or
- there is a path from i to j going through vertex 1 and/or 2; or
- there is a path from i to j going through vertex 1, 2, and/or 3; or
- ...
- there is a path from i to j going through any of the other vertices



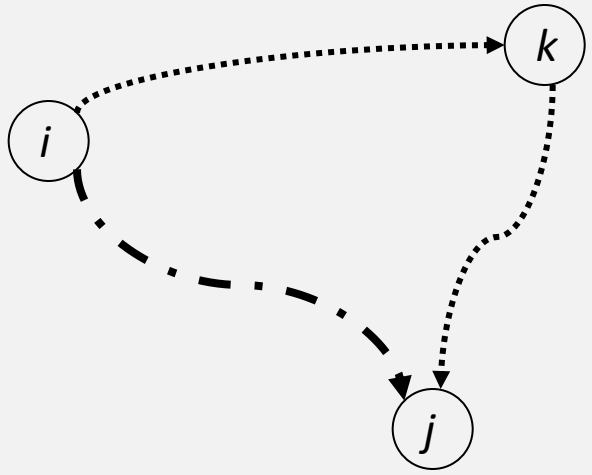
$R^{(0)}$	$R^{(1)}$	$R^{(2)}$	$R^{(3)}$	$R^{(4)}$
0 0 1 0	0 0 1 0	0 0 1 0	0 0 1 0	0 0 1 0
1 0 0 1	1 0 **1** 1	1 0 1 1	1 0 1 1	1 **1** 1 1
0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0
0 1 0 0	0 1 0 0	**1** 1 **1** **1**	1 1 1 1	1 1 1 1

Dynamic programming

Warshall's Algorithm: Recurrence

- On the k-th iteration, the algorithm determines for every pair of vertices i, j if a path exists from i and j with just vertices 1,…,k allowed as intermediate

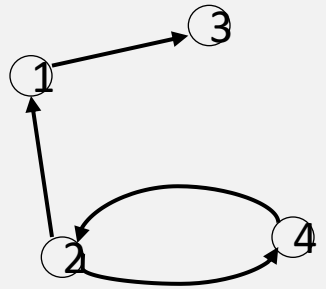
- $$R^{(k)}[i,j] = \begin{cases} R^{(k-1)}[i,j] & \text{(path using just 1 ,…,k−1)} \\ R^{(k-1)}[i,k] \text{ and } R^{(k-1)}[k,j] & \text{(path from i to k and from k to j using just 1 ,…,k−1)} \end{cases}$$



- If an element r_{ij} is 1 in $R^{(k-1)}$, it remains 1 in $R^{(k)}$.
- If an element r_{ij} is 0 in $R^{(k-1)}$, it has to be changed to 1 in $R^{(k)}$ if and only if the element in its row i and column k and the element in its column j and row k are both 1's in $R^{(k-1)}$

Dynamic programming

Warshall's Algorithm: Example



$R^{(0)} =$

0	0	1	0
1	0	0	1
0	0	0	0
0	1	0	0

$R^{(1)} =$

0	0	1	0
1	0	**1**	1
0	0	0	0
0	1	0	0

$R^{(2)} =$

0	0	1	0
1	0	1	1
0	0	0	0
1	1	**1**	**1**

$R^{(3)} =$

0	0	1	0
1	0	1	1
0	0	0	0
1	1	1	1

$R^{(4)} =$

0	0	1	0
1	**1**	1	1
0	0	0	0
1	1	1	1

Dynamic programming

Warshall's Algorithm: Algorithm Analysis

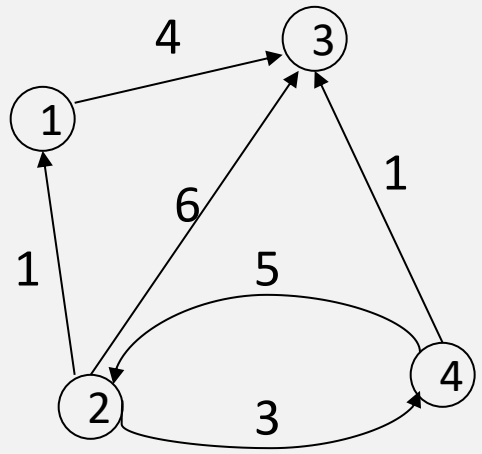
ALGORITHM *Warshall(A[1..n, 1..n])*
//ImplementsWarshall's algorithm for computing the transitive closure
//Input: The adjacency matrix *A* of a digraph with *n* vertices
//Output: The transitive closure of the digraph
$R_{(0)} \leftarrow A$
for $k \leftarrow 1$ **to** n **do**
    for $i \leftarrow 1$ **to** n **do**
        for $j \leftarrow 1$ **to** n **do**
            $R_{(k)}[i, j] \leftarrow R_{(k-1)}[i, j]$ **or** $(R_{(k-1)}[i, k]$ **and** $R_{(k-1)}[k, j])$
return $R_{(n)}$

Time efficiency: (n^3)
Space efficiency: Matrices can be written over their predecessors

Dynamic programming

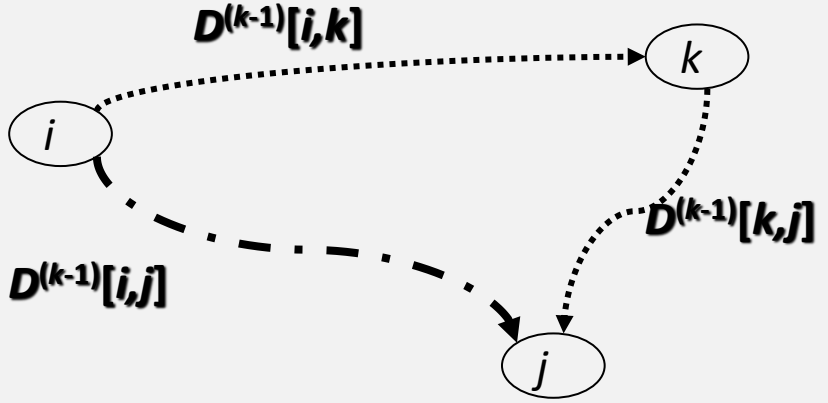
Floyd's Algorithm: All pairs shortest paths

- **Problem:** In a weighted (di)graph, find shortest paths between every pair of vertices
- **Same idea:** construct solution through series of matrices $D^{(0)}$, …, $D^{(n)}$ using increasing subsets of the vertices allowed as intermediate.
- **Example:**



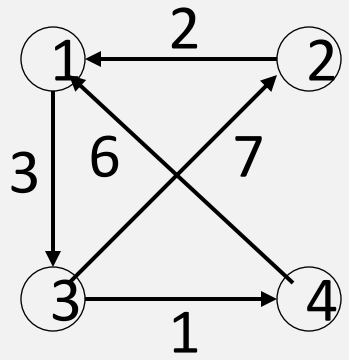
On the k-th iteration, the algorithm determines shortest paths between every pair of vertices i, j that use only vertices among 1,…,k as intermediate

$$D^{(k)}[i,j] = \min \{D^{(k-1)}[i,j],\ D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\}$$



Dynamic programming

Floyd's Algorithm: All pairs shortest paths



$D^{(0)} =$

0	∞	3	∞
2	0	∞	∞
∞	7	0	1
6	∞	∞	0

$D^{(1)} =$

0	∞	3	∞
2	0	**5**	∞
∞	7	0	1
6	∞	**9**	0

$D^{(2)} =$

0	∞	3	∞
2	0	5	∞
9	7	0	1
6	∞	9	0

$D^{(3)} =$

0	**10**	3	**4**
2	0	5	**6**
9	7	0	1
6	**16**	9	0

$D^{(4)} =$

0	10	3	4
2	0	5	6
7	7	0	1
6	16	9	0

Dynamic programming

Floyd's Algorithm: Algorithm analysis

ALGORITHM *Floyd(W[1..n, 1..n])*
//Implements Floyd's algorithm for the all-pairs shortest-paths problem
//Input: The weight matrix *W* of a graph with no negative-length cycle
//Output: The distance matrix of the shortest paths' lengths
$D \leftarrow W$ //is not necessary if *W* can be overwritten
for $k \leftarrow 1$ **to** n **do**
    for $i \leftarrow 1$ **to** n **do**
        for $j \leftarrow 1$ **to** n **do**
            $D[i, j] \leftarrow \min\{D[i, j], D[i, k] + D[k, j]\}$
return *D*

Time efficiency: (n^3)
Space efficiency: Matrices can be written over their predecessors